True states remain reasonable in the finite-rank tuple-recovery worker construction

David Gonzalez, Matthew Harrison-Trainor, Julia Knight

arxiv:2610.05611v1; CC BY 4.0

Editorial note. Complete active original author body retained up to its bibliography; original text after end-document is not part of the delivered article. Exactly one primary per task. No proof repair, added assumptions or correctness certification.

Editorial note. Mechanical bibliography adapter protects only unescaped original math expressions against BibTeX title-case conversion; original Reference.bib retained.

Allowed standard prerequisites (editorial).

- Computable structures, back-and-forth ranks and Ash metatheorems. Native ELL-rank, labelled tuple-tree and ordinal genericity definitions plus the full finite-level worker apparatus retained.. Finite-n reconstruction is selected with its complete original proof; later transfinite sketch cases and hardness statements are not counted as separate targets.

1 Introduction

In their seminal paper [6], Friedman and Stanley showed that the equivalence relation of isomorphism on trees is as complicated as the equivalence relation of isomorphism for structures in any finite relational language $\mathcal{L}$ by giving a Borel embedding that takes each countable $\mathcal{L}$-structure $\mathcal{A} \in Mod(\mathcal{L})$ to a tree $T_{\mathcal{A}}$ such that

$$\mathcal{A} \cong \mathcal{B} \iff T_{\mathcal{A}} \cong T_{\mathcal{B}}.$$

In fact, the embedding is Turing computable. In [13], $T_{\mathcal{A}}$, called the *tree of tuples* of $\mathcal{A}$, is described as a labeled tree whose nodes are the finite tuples of $\mathcal{A}$, ordered by extension (tuples of length n are at level n), with each tuple labeled by its atomic diagram. This construction has been used as a step in the proof of several further Borel complexity results, including the analysis of structures as diverse as linear orderings [6], Boolean algebras [3], and models of Peano arithmetic [18, 8].

The tree of tuples construction does not have the regularity and preservation properties enjoyed by many of the commonly studied Borel embeddings. For example, by results of Gao [7] (see also [10] and [12]), the class $\mathbb{T}$ of labeled trees isomorphic to those in the range of the embedding is not Borel. In [9], Gonzalez

and Knight considered subclasses $\mathbb{T}^\alpha$ consisting of those $T \cong T_\mathcal{A}$ for $\mathcal{A}$ of Scott rank at most α. Their result, contrasting with that of Gao, says that if α is an X-computable ordinal, then the class $\mathbb{T}^\alpha$ is X-effectively $\Pi^0_{2\alpha+2}$-complete. In [19], Montalbán defined the *Scott rank* of $\mathcal{A}$ to be the least ordinal α such that $\mathcal{A}$ has a $\Pi_{\alpha+1}$ Scott sentence, or equivalently, such that the orbits of all tuples in $\mathcal{A}$ are defined by Σ_α-formulas (Montalbán gave several further equivalent conditions). This is the definition of Scott rank used in [9].

In the present paper, we consider the following question: Given a labeled tree T that is isomorphic to $T_\mathcal{A}$ for some $\mathcal{L}$-structure $\mathcal{A}$, how hard is it to compute a copy of $\mathcal{A}$? Harrison-Trainor and Montalbán showed in [13] that there is a structure $\mathcal{A}$ such that $T_\mathcal{A}$ has a computable copy, but $\mathcal{A}$ has no computable copy. Moreover, Harrison-Trainor and Montalbán showed[1] that for each computable ordinal $\alpha \geq 2$, there is a structure $\mathcal{A}$ such that $T_\mathcal{A}$ has a computable copy, but $\mathcal{A}$ has no Δ^0_α copy. Relativizing this to any oracle X, we see that there is no $\mathbf{\Delta}^0_\alpha$ operator, for any ordinal $\alpha \geq 2$, which can uniformly recover structures from their tree of tuples.[2]

The work of Gonzalez and Knight [9] mentioned above suggests that the level of difficulty in recovering a structure from its tree of tuples is connected to the Scott rank of the structure. It is quite easy to show that if $\mathcal{A}$ has Scott rank 1, meaning that the automorphism orbit of each tuple is defined by an existential formula, then given a copy of $T_\mathcal{A}$, we can computably build a copy of $\mathcal{A}$. The proof is given in Case 1 of Section 3. On the other hand, the structure $\mathcal{A}$ constructed by Harrison-Trainor and Montalbán (such that $T_\mathcal{A}$ has a computable copy, while $\mathcal{A}$ does not) can easily be seen to have Scott rank 2. A naive argument using the machinery of [9] shows that for each computable ordinal $\alpha = \beta + 1$, there is an effective $\Delta^0_{2\beta+1}$ operator that, for all $\mathcal{A}$ of rank α, takes copies of $T_\mathcal{A}$ to copies of $\mathcal{A}$. Using a worker construction, we prove something much stronger.

Theorem 1.1. *For each computable ordinal $\alpha \geq 1$, there is an effective Δ^0_α operator that, for each $\mathcal{A} \in Mod(\mathcal{L})$ of Scott rank at most α (or even* ELL*-rank at most α), takes copies of $T_\mathcal{A}$ to copies of $\mathcal{A}$.*

ELL-rank, which we define in Section 2.2, is a new variant of Scott rank based on the symmetric back-and-forth relations $\equiv_\alpha$ considered by Ehrenfeucht [5] and Läuchli and Leonard [16]. If a structure has Scott rank at most α, then it also has ELL-rank at most α, but not vice versa, so the theorem stated in terms of ELL-rank is stronger. Theorem 1.1 relativizes. For each X and each X-computable ordinal $\alpha \geq 1$, there is an X-effective Δ^0_α operator that, for each $\mathcal{A} \in Mod(\mathcal{L})$ of ELL-rank at most α, takes copies of $T_\mathcal{A}$ to copies of $\mathcal{A}$. Thus, we get the following corollary.

Corollary 1.2. *For each countable ordinal $\alpha \geq 1$, there is a $\mathbf{\Delta}^0_\alpha$ operator that, for each $\mathcal{A} \in Mod(\mathcal{L})$ of Scott rank at most α (or even* ELL*-rank at most α), takes copies of $T_\mathcal{A}$ to copies of $\mathcal{A}$.*

We also give a partial hardness result saying that at finite levels, this cannot be improved. For finite $n > 1$, there is no $\mathbf{\Delta}^0_{n-1}$ operator that works even

[1] The published proof in [13] is incorrect. See the erratum [14].

[2] By a $\mathbf{\Delta}^0_\alpha$ operator we mean a $\mathbf{\Delta}^0_\alpha$-measurable function between spaces of structures, and by an effective Δ^0_α operator we mean a $\mathbf{\Delta}^0_\alpha$ operator with a computable code.

for structures of Scott rank at most n (and hence no operator that works for structures of ELL-rank at most n). This argument uses the unfriendly jump inversion of Harrison-Trainor [11].

In the proof of Theorem 1.1, we define objects like those in Ash's metatheorem. We have an alternating tree, a family of back-and-forth relations, etc., forming an α-system. For us, α will be finite. We also have an instruction function, providing high-level information. What prevented us from simply applying Ash's metatheorem is the fact that these back-and-forth relations are not all c.e. relative to the appropriate oracle. The proof of Ash's metatheorem given in [2] used workers,[3] and we use workers here. The new idea is that the instructions are changed as they are passed down. In consequence, Worker β needs only to enumerate up to the βth back-and-forth relation. To make this work, we found it necessary to use the symmetric back-and-forth relations $\equiv_\beta$, and not the standard back-and-forth relations $\leq_\beta$. We believe that there may be other applications of this method. See Subsection 3.2.1 for further discussion.

2 Preliminaries

2.1 Back-and-forth relations

Definition 2.1. Let $\mathcal{L}$ be a finite relational language, and let $\mathcal{A} \in Mod(\mathcal{L})$. Suppose $\bar{a}$ and $\bar{b}$ are tuples in $\mathcal{A}$ of the same length.

(1) $\bar{a} \equiv_0 \bar{b}$ if $\bar{a}$ and $\bar{b}$ satisfy the same quantifier-free type,

(2) for $\alpha > 0$, $\bar{a} \equiv_\alpha \bar{b}$ if for all $\beta < \alpha$,

- for each $\bar{c}$ there exists $\bar{d}$ such that $\bar{a}, \bar{c} \equiv_\beta \bar{b}, \bar{d}$, and
- for each $\bar{d}$ there exists $\bar{c}$ such that $\bar{a}, \bar{c} \equiv_\beta \bar{b}, \bar{d}$.

The original proof of the Scott Isomorphism Theorem [20] involved symmetric relations, although the added tuples $\bar{c}$ and $\bar{d}$ were single elements. Ehrenfeucht [5] and Läuchli and Leonard [16] used tuples. Their equivalence relations were exactly as above.

Often in computable structure theory, one uses instead the asymmetric relations defined as follows.

Definition 2.2. Let $\mathcal{L}$ be a finite relational language, and let $\mathcal{A} \in Mod(\mathcal{L})$. Suppose $\bar{a}$ and $\bar{b}$ are tuples in $\mathcal{A}$ of the same length.

(1) $\bar{a} \leq_0 \bar{b}$ if $\bar{a}$ and $\bar{b}$ satisfy the same quantifier-free type,

(2) for $\alpha > 0$, $\bar{a} \leq_\alpha \bar{b}$ if for all $\beta < \alpha$ and $\bar{c}$ there exists $\bar{d}$ such that $\bar{b}, \bar{c} \leq_\beta \bar{a}, \bar{d}$.

What makes these asymmetric relations useful is the fact, proved by Karp [15], that $\bar{a} \leq_\alpha \bar{b}$ if and only if every Σ_α formula true of $\bar{b}$ is true of $\bar{a}$. Often one will see $\equiv_\alpha$ defined by $\bar{a} \equiv_\alpha \bar{b}$ if and only if $\bar{a} \leq_\alpha \bar{b}$ and $\bar{b} \leq_\alpha \bar{a}$. These equivalence relations are quite different from the ones in Definition 2.1. When we use the notation in this paper, we always intend the meaning in Definition

[3] Ash's original proof of the metatheorem did not use workers; it was inspired by Martin's proof of Borel Determinacy.

2.1, never the other possible meaning. We are describing the *true* symmetric back-and-forth relations, rather than the symmetrization of the asymmetric relations. The precise reasons behind our use of the symmetric relations are somewhat technical, but we offer at least some explanation in Remark 2.11 and Subsection 3.2.1. Our symmetric relations still satisfy the following.

Lemma 2.3. *If $\bar{a} \equiv_\beta \bar{a}'$, then $\bar{a} \leq_\beta \bar{a}'$ and $\bar{a}' \leq_\beta \bar{a}$.*

Proof. This is a straightforward induction on β. □

Remark 2.4. For X-computable ordinals β, there are X-computable $\Pi_{2\beta}$ formulas that, in all $\mathcal{L}$-structures, define the relations $\leq_\beta$. The same is true for $\equiv_\beta$. In [4], it is shown that there is no simpler definition of the relations $\leq_\beta$, and, although we do not have a proof, we believe that this is true for the relations $\equiv_\beta$ as well.

Remark 2.5. We often define the back-and-forth relations $\bar{a} \leq_\alpha \bar{b}$ or $\bar{a} \equiv_\alpha \bar{b}$ between tuples $\bar{a}$ and $\bar{b}$ of different lengths, with $\bar{b}$ being of greater length than $\bar{a}$. This is just saying that the relation holds between $\bar{a}$ and the initial segment of $\bar{b}$ of the same length as $\bar{a}$.

2.2 Scott rank and ELL-rank

While historically there have been numerous inequivalent definitions of Scott rank, in this paper we use Montalbán's Scott rank introduced in [19]. This rank is robust because, unlike the other definitions, it can be equivalently defined in several different ways. Montalbán demonstrated the equivalence of several conditions. Below, we give just three of these conditions.

Theorem 2.6 ([19])**.** *For a structure $\mathcal{A} \in Mod(\mathcal{L})$ and a countable ordinal $\alpha \geq 1$, the following are equivalent:*

(1) $\mathcal{A}$ has a $\Pi_{\alpha+1}$ Scott sentence,

(2) for all tuples $\bar{a}$ in $\mathcal{A}$, the orbit is defined by a Σ_α formula,

(3) for each tuple $\bar{a}$, there is a Σ_α formula true of $\bar{a}$ that (in $\mathcal{A}$) implies all Π_α formulas true of $\bar{a}$ in $\mathcal{A}$.

Definition 2.7. The *Scott rank* of $\mathcal{A}$ is the least ordinal α satisfying the conditions of Montalbán's Theorem.

The following observation is given in [9].

Proposition 2.8. *$\mathcal{A}$ has Scott rank α iff α is least such that the following equivalent conditions hold:*

(1) for each $\bar{a}$, there exist $\beta < \alpha$ and $\bar{b}$ such that for all $\bar{a}', \bar{b}'$, $\bar{a}, \bar{b} \leq_\beta \bar{a}', \bar{b}'$ implies $\bar{a} \leq_\alpha \bar{a}'$,

(2) for each $\bar{a}$, there exist $\beta < \alpha$ and $\bar{b}$ such that for all $\bar{a}', \bar{b}'$, $\bar{a}, \bar{b} \leq_\beta \bar{a}', \bar{b}'$ implies $\bar{a} \cong \bar{a}'$.

For the symmetric relations $\equiv_\beta$, we have the analogous result.

Proposition 2.9. *For a countable ordinal α and $\mathcal{A} \in Mod(\mathcal{L})$, the following are equivalent:*

(1) for each $\bar{a}$, there exist $\beta < \alpha$ and $\bar{b}$ such that for all $\bar{a}'$ and $\bar{b}'$, $\bar{a}, \bar{b} \equiv_\beta \bar{a}', \bar{b}'$ implies $\bar{a} \equiv_\alpha \bar{a}'$,

(2) for each $\bar{a}$, there exist $\beta < \alpha$ and $\bar{b}$ such that for all $\bar{a}'$ and $\bar{b}'$, $\bar{a}, \bar{b} \equiv_\beta \bar{a}', \bar{b}'$ implies $\bar{a} \cong \bar{a}'$.

Proof. It is clear that (2) implies (1). Supposing (1), we prove (2). It is enough to show that the family of functions $\bar{a} \to \bar{a}'$ such that $\bar{a} \equiv_\alpha \bar{a}'$ has the back-and-forth property. Suppose $\bar{a} \equiv_\alpha \bar{a}'$. Given c, we need c' such that $\bar{a}, c \equiv_\alpha \bar{a}', c'$. By assumption, there exist $\beta < \alpha$ and $\bar{b}$ such that $\bar{a}, c, \bar{b} \equiv_\beta \bar{a}', c', \bar{b}'$ implies $\bar{a}, c \equiv_\alpha \bar{a}', c'$. Since $\bar{a} \equiv_\alpha \bar{a}'$, there exist c' and $\bar{b}'$ such that $\bar{a}, c, \bar{b} \equiv_\beta \bar{a}', c', \bar{b}'$. Then $\bar{a}, c \equiv_\alpha \bar{a}', c'$, as required. We also need to show that given c', there exists c such that $\bar{a}, c \equiv_\alpha \bar{a}', c'$, but this follows from the symmetry of the relation $\equiv_\alpha$. □

Here is the modified rank.

Definition 2.10. An $\mathcal{L}$-structure $\mathcal{A}$ has ELL-*rank* α if α is least such that the conditions in Proposition 2.9 hold.

Remark 2.11. This differs slightly from the definition saying that the rank of an $\mathcal{L}$-structure $\mathcal{A}$ is the least α such that for all tuples $\bar{a}, \bar{b}$ we have

$$\bar{a} \equiv_\alpha \bar{b} \Longrightarrow \bar{a} \equiv_{\alpha+1} \bar{b}.$$

That notion of Scott rank is the one called $SR(\mathcal{A})$ in the book by Ash and Knight [2]. We use ELL-rank because it is precisely the rank that arises from the proof of Theorem 1.1 and, hence, gives the optimal result.

2.3 The tree of tuples

Let $\mathcal{L}$ be a relational language. For simplicity, we suppose that it is finite.[4] We include in our language the logical constant $\top$ (truth). Let $\mathcal{D}$ be the set of $\mathcal{L}$-formulas $D(x_1, \ldots, x_n)$ formed as the conjunction of formulas saying either $R(\bar{x})$ or $\neg R(\bar{x})$, for each relation symbol $R \in \mathcal{L}$ and each tuple $\bar{x}$ from among $x_1, \ldots, x_n$. We include in $\mathcal{D}$ the formula $\top$ (truth), with the empty sequence of variables. To each $\mathcal{A} \in Mod(\mathcal{L})$, we assign a labeled tree $T_\mathcal{A}$, the *tree of tuples*. The language of the labeled trees consists of a unary function symbol p, for the predecessor function (which has the convention $p(root) = root$), and unary relation symbols U_D for $D \in \mathcal{D}$, for the labels. The tree itself will be isomorphic to $\omega^{<\omega}$. Each node σ of the tree $T_\mathcal{A}$ *represents* a tuple $a_1, \ldots, a_n$ of distinct elements of $\mathcal{A}$, with each non-empty tuple showing up infinitely many times. The node σ is in a single unary relation U_D, where $\mathcal{A} \models D(a_1, \ldots, a_n)$.

Definition 2.12. For $\mathcal{A} \in Mod(\mathcal{L})$, the nodes of $T_\mathcal{A}$ are defined by induction on the level as follows:

[4] To justify this, we may apply a result of Lavrov [17] saying that for any countable language $\mathcal{L}$, $Mod(\mathcal{L})$ is Borel reducible to the class of undirected graphs, and if $\mathcal{L}$ is computable, then the reduction is Turing computable.

(1) At level 0, the root node, representing the empty tuple, has the label $U_\top$.

(2) Suppose σ is a node at level n, representing an n-tuple $\bar{a}$ from $\mathcal{A}$, and with label U_D, where D gives the atomic type of $\bar{a}$. Let a' be an element of $\mathcal{A}$ not in $\bar{a}$. Then σ has infinitely many successors σ' representing the tuple $\bar{a}, a'$, with the appropriate label $U_{D'}$, where $D'(x_1, \ldots, x_n, x_{n+1})$ gives the atomic type of the $(n+1)$-tuple $\bar{a}, a'$.

The tree of tuples captures many aspects of the structure, the most important being the back-and-forth structure. Indeed, this is the reason that

$$T_\mathcal{A} \cong T_\mathcal{B} \Longrightarrow \mathcal{A} \cong \mathcal{B}.$$

If $T_\mathcal{A} \cong T_\mathcal{B}$ then using a back-and-forth procedure, we can arrive at an isomorphism between a copy of $\mathcal{A}$ and a copy of $\mathcal{B}$, each represented by a path through the appropriate tree. We make the following definition to capture the back-and-forth relations on the tree $T_\mathcal{A}$. In the following definition, we use the notation $\sigma \succeq \tau$ for $\sigma, \tau \in T_\mathcal{A}$ to mean that $p^n(\sigma) = \tau$ for some n. In $T_\mathcal{A}$, the relation $\sigma \succeq \tau$ is defined by a computable Σ_1 formula. This shows that the relation $\sigma \succeq \tau$ is uniformly c.e. relative to the tree. In fact, it is computable relative to the tree. To see this, note that $\sigma \succeq \tau$ iff τ is among the finite set of predecessors of σ, which we can effectively list.

Definition 2.13. If σ is at level n in $T_\mathcal{A}$, we say that $x_1, \ldots, x_n$ are the variables of σ. Let $\sigma, \tau \in T_\mathcal{A}$, and let $\bar{x}, \bar{y}$ be tuples of variables of the same length, where $\bar{x}$ is among the variables of σ and $\bar{y}$ is among the variables of τ. Then

(1) $(\sigma, \bar{x}) \equiv_0 (\tau, \bar{y})$ if σ assigns to $\bar{x}$ the same quantifier-free type that τ assigns to $\bar{y}$.

(2) for $\alpha > 0$, $(\sigma, \bar{x}) \equiv_\alpha (\tau, \bar{y})$ if for all $\beta < \alpha$,

- for each $\sigma' \succeq \sigma$ and each $\bar{x}'$ (among the variables of σ'), there exist $\tau' \succeq \tau$ and $\bar{y}'$ (among the variables of τ') such that $(\sigma', \bar{x}\bar{x}') \equiv_\beta (\tau', \bar{y}\bar{y}')$, and
- for each $\tau' \succeq \tau$ and each $\bar{y}'$ (among the variables of τ'), there exist $\sigma' \succeq \sigma$ and $\bar{x}'$ (among the variables of σ') such that $(\sigma', \bar{x}\bar{x}') \equiv_\beta (\tau', \bar{y}\bar{y}')$.

We can make a similar definition for the asymmetric back-and-forth relations. Like the back-and-forth relations on $\mathcal{A}$ itself, the relations $\equiv_\alpha$ on $T_\mathcal{A}$ are $\Pi^0_{2\alpha}$.

In $T_\mathcal{A}$, each node represents a tuple from $\mathcal{A}$. Suppose σ represents $(a_1, \ldots, a_r)$ and τ represents $(b_1, \ldots, b_s)$ —we have that

$$(\sigma, (x_{i_1}, \ldots, x_{i_n})) \equiv_\alpha (\tau, (y_{j_1}, \ldots, y_{j_n}))$$

if and only if $a_{i_1}, \ldots, a_{i_n} \equiv_\alpha b_{j_1}, \ldots, b_{j_n}$. The advantage of the definition above is that it is independent of the identification of nodes in $T_\mathcal{A}$ with tuples of $\mathcal{A}$ and hence is independent of the copy of $\mathcal{A}$.

2.4 Generic paths

For an infinite $\mathcal{A} \in Mod(\mathcal{L})$, and σ at level n of $T_{\mathcal{A}}$, the label assigns a quantifier-free type to the variables $x_1, \ldots, x_n$, making all of the elements distinct. Each path p through T yields a structure $\mathcal{A}^p$, with universe equal to the full set of variables $\{x_1, x_2, \ldots\}$. The structure $\mathcal{A}^p$ is always isomorphic to a substructure of $\mathcal{A}$, and, in fact, we get copies of all infinite substructures of $\mathcal{A}$ this way. Each node σ extends to a path p such that $\mathcal{A}^p \cong \mathcal{A}$. We want, just looking at $T \cong T_{\mathcal{A}}$, to construct a path p that we can be sure will yield a copy of $\mathcal{A}$.

In [13], it is noted that a sufficiently generic path will yield a copy of $\mathcal{A}$. In [9], we define a forcing notion and show that if $\mathcal{A}$ has a Π_α Scott sentence, and p is a generic path through $T_{\mathcal{A}}$, then $\mathcal{A}^p \cong \mathcal{A}$. Also in [9], we define, without reference to forcing, what it means for a path p to be "α-generic," and we show that for $\mathcal{A}$ of Scott rank at most α, if p is an α-generic path through $T_{\mathcal{A}}$, then $\mathcal{A}^p \cong \mathcal{A}$. We will define what it means for a path to be ELL-α-generic, and we show that for $\mathcal{A}$ of ELL-rank at most α, if p is an ELL-α-generic path through $T_{\mathcal{A}}$, then $\mathcal{A}^p \cong \mathcal{A}$. We begin with the definition of α-genericity from [9].

Definition 2.14. Let $T \cong T_{\mathcal{A}}$, and let $\alpha \geq 1$ be a countable ordinal. A path p through T is *α-generic* provided that for all $\beta < \alpha$ and all pairs of nodes (ν, τ) such that $\nu \prec p$ and $\nu \prec \tau$, where ν has variables $\bar{x}$, and τ has variables $\bar{x}, \bar{y}$, there is some τ', with variables including $\bar{x}$ and some $\bar{z}$, such that $\nu \prec \tau' \prec p$ and $(\tau, \bar{x}, \bar{y}) \leq_\beta (\tau', \bar{x}, \bar{z})$.

Here is the analogous definition, with $\equiv_\beta$ replacing $\leq_\beta$.

Definition 2.15. Let $T \cong T_{\mathcal{A}}$, and let $\alpha \geq 1$ be a countable ordinal. A path p through T is ELL-*α-generic* provided that for all $\beta < \alpha$ and all pairs of nodes (ν, τ) such that $\nu \prec p$ and $\nu \prec \tau$, where ν has variables $\bar{x}$, and τ has variables $\bar{x}, \bar{y}$, there is some τ', with variables including $\bar{x}$ and some $\bar{z}$, such that $\nu \prec \tau' \prec p$ and $(\tau, \bar{x}, \bar{y}) \equiv_\beta (\tau', \bar{x}, \bar{z})$.

Here is the analogue of the result from [9].

Proposition 2.16. *Suppose $\mathcal{A} \in Mod(\mathcal{L})$ has* ELL-*rank at most α, and let p be an* ELL-*α-generic path through $T \cong T_{\mathcal{A}}$. Then $\mathcal{A}^p \cong \mathcal{A}$.*

Proof. Since the notion of an ELL-α-generic path is invariant under isomorphisms, we may assume that $T = T_{\mathcal{A}}$. Let $\mathcal{F}$ be the set of finite partial isomorphisms f such that for some $\sigma \prec p$, with variables $\bar{x}$, f maps $\bar{x}$ to $\bar{a} \in \mathcal{A}$ and σ represents a tuple in the orbit of $\bar{a}$. We show that $\mathcal{F}$ has the back-and-forth property. Clearly, $\mathcal{F}$ is non-empty.

Forth: Suppose $\bar{x} \to \bar{a}$ is in $\mathcal{F}$. Say $\sigma \prec p$ has variables $\bar{x}$ and represents a tuple $\bar{a}'$ such that $\bar{a}' \cong \bar{a}$. Let y be a variable not in $\bar{x}$—for simplicity, let it be the next variable. Let σ' be the successor of σ with $\sigma' \prec p$. Then σ' represents some $\bar{a}', c'$, and the automorphism taking $\bar{a}$ to $\bar{a}'$ takes some c to c'. Then $\bar{x}, y \to \bar{a}, c$ is in $\mathcal{F}$.

Back: Suppose $f \in \mathcal{F}$, where f takes $\bar{x}$ to $\bar{a}$. Say $\sigma \prec p$ has variables $\bar{x}$ and represents a tuple $\bar{a}'$ in the orbit of $\bar{a}$. Let c be an element of $\mathcal{A}$ not in $\bar{a}$. We have c' such that $\bar{a}, c \cong \bar{a}', c'$. There exist $\bar{b}'$ and $\beta < \alpha$ such that $\bar{a}', c', \bar{b}' \equiv_\beta \bar{a}'', c'', \bar{b}''$ implies $\bar{a}', c' \cong \bar{a}'', c''$. Take τ, with variables $\bar{x}, y, \bar{u}$ such that $\sigma \prec \tau$, and τ represents $\bar{a}', c', \bar{b}'$. By ELL-α-genericity, there exists τ' with $\sigma \prec \tau' \prec p$ with

variables including $\bar{x}$ and some z and $\bar{v}$ such that $(\tau, \bar{x}, y, \bar{u}) \equiv_\beta (\tau', \bar{x}, z, \bar{v})$. Then τ' represents a tuple that includes $\bar{a}'$ and some $c'', \bar{b}''$ corresponding to z and $\bar{v}$, and $\bar{a}, c \cong \bar{a}', c''$. Therefore, the function f' taking $\bar{x}, z$ to $\bar{a}, c$ is in $\mathcal{F}$. □

Remark 2.17. By examining the alternations of quantifiers present in the definition of ELL-α-generic, we can bound the complexity of an operator that, given $T \in \mathbb{T}$, produces an α-generic path. If α is an X-computable limit ordinal, then we may take the operator to be X-effective Δ^0_α, and if $\alpha = \beta + 1$ is an X-computable successor ordinal, then we may take the operator to be X-effective $\Delta^0_{2\beta+1}$.

3 Proof of Theorem 1.1

In this section, we prove Theorem 1.1. We recall the statement below.

Theorem 1.1. Let $X \subseteq \omega$, and suppose α is an X-computable ordinal. Then there is an X-effective Δ^0_α operator that, for structures $\mathcal{A}$ of ELL-rank at most α, takes copies of $T_\mathcal{A}$ to copies of $\mathcal{A}$.

We split the proof of Theorem 1.1 into the following cases:

(1) $\alpha = 1$,

(2) $\alpha = n > 1$ finite,

(3) α is a limit ordinal,

(4) $\alpha = \lambda + n$, where λ is a limit ordinal and $n \geq 1$ is finite.

We begin with the case $\alpha = 1$, which is relatively simple but illustrates the general method. We then follow with the case of $\alpha = n > 1$ finite, which is the key case. The case of α a limit ordinal is easy, since $2\alpha = \alpha$. Finally, the method for finite n can easily be adapted to the case $\alpha = \lambda + n$.

3.1 The case $\alpha = 1$

In this subsection, we prove Theorem 1.1 in Case 1, where $\alpha = 1$.

Case 1. *There is a Turing computable operator Φ such that if $T \cong T_\mathcal{A}$ and $\mathcal{A}$ is of* ELL*-rank at most* 1*, then $\Phi(T) \cong \mathcal{A}$.*

Proof. It is enough to give an effective procedure that, given such a T, produces an ELL-1-generic path p. Say that $T \cong T_\mathcal{A}$, where $\mathcal{A}$ has ELL-rank at most 1. By Proposition 2.16, $\mathcal{A}^p \cong \mathcal{A}$. It is clear that we can pass effectively from p to the atomic diagram of $\mathcal{A}^p$—the label on the node at level n gives the atomic type of $(x_1, \ldots, x_n)$.

Computably in T, we can list the pairs (ν, τ) such that $\nu \prec \tau$. We satisfy the following requirements:

R_n: For the nth pair (ν, τ) with $\nu \prec \tau$, where ν has variables $\bar{x}$ and τ has variables $\bar{x}, \bar{y}$, and τ assigns the atomic type $D(\bar{x}, \bar{y})$, if $\nu \prec p$, then there exists τ', with variables including $\bar{x}$ and some $\bar{z}$, such that $\nu \prec \tau' \prec p$ and $(\tau, \bar{x}, \bar{y}) \equiv_0 (\tau', \bar{x}, \bar{z})$.

At each stage s, we have determined a node σ_s on p. We satisfy the first requirement that needs attention. At stage $s+1$, consider requirements R_n for $n \leq s+1$, and (ν, τ) among the first s on our list. The requirement needs attention if $\nu \prec \sigma_s$, and there is no $\tau' \prec \sigma_s$ witnessing the appropriate atomic type. If we do not find a requirement needing attention, then σ_{s+1} is just some successor of σ_s. Assuming that there is a requirement R_n needing attention, we take the first. There is some extension τ'' of σ_s satisfying the requirement, since τ and σ_s extend to paths that yield copies of $\mathcal{A}$. Knowing that there is a node satisfying τ'', we can effectively find one, and we let this be σ_{s+1}. The resulting path p satisfies the witnessing requirements, and is therefore ELL-1-generic. □

3.2 The case α finite, greater than 1

In this subsection, we prove Theorem 1.1 in Case 2, where α is finite, greater than 1.

Case 2. *For each finite $n > 1$, there is an effective Δ^0_n operator that, for all $\mathcal{A}$ of* ELL*-rank at most n, maps copies of $T_{\mathcal{A}}$ to copies of $\mathcal{A}$.*

Proof. We describe an effective Δ^0_n procedure that, for each $T \cong T_{\mathcal{A}}$ with $\mathcal{A}$ of ELL-rank at most n, gives a structure $\mathcal{B}$ (with universe ω) that is isomorphic to $\mathcal{A}^p$ for some ELL-n-generic path p. We use a worker construction, with n workers, where for $1 \leq k \leq n$, Worker k has oracle $\Delta^0_k(\Delta^0_n(T)) = \Delta^0_{n+k-1}(T)$. In describing the construction, we suppress the oracle T. We have already defined $\equiv_k$ on pairs $(\sigma, \bar{x})$, where σ is a node in T and $\bar{x}$ is among the variables of σ. As we will see in Fact 3.3 below, for $1 \leq k \leq n$, Worker k can compute $\equiv_{k-1}$, and for $1 \leq k \leq n-2$, Worker k can also compute $\equiv_k$.

3.2.1 Workers and the metatheorem of Ash

Ash proved a metatheorem, with conditions on some abstract objects, guaranteeing the existence of a c.e. set that satisfies some high-level requirements. There are a number of applications of the metatheorem in computable structure theory. Although the metatheorem does not require it, most applications use the standard asymmetric back-and-forth relations. While the original proof of the metatheorem did not use workers, the proof in [2] does. We say a little about the proof from [2], in the case where there are just n workers, since our proof will follow a similar outline. For $1 \leq k \leq n$, Worker k uses the oracle Δ^0_k; throughout, the construction may be relativized. Worker n determines an infinite sequence $\ell_0 u^n_1 \ell^n_1 u^n_2 \ell^n_2 \ldots$ that is a path through a c.e. "alternating" tree P, where the terms u^n_s are given by a Δ^0_n "instruction" function. There is a computable "enumeration" function E that, for each possible ℓ, enumerates a set. The goal of the construction is for $\cup_s E(\ell^n_s)$ to be c.e.

For $1 \leq k < n$, Worker k produces a sequence, based on guesses at the sequence produced by Worker $k+1$. The sequence ℓ^n_s produced by Worker n is a subsequence of that produced by Worker 1. For $k \leq n$, Worker k preserves $\leq_{k-1}$; that is, $\ell^k_m \leq_{k-1} \ell^k_{m+1}$. Since $\ell \leq_k \ell'$ implies $\ell \leq_{k-1} \ell'$, and $\ell \leq_0 \ell'$ implies $E(\ell) \subseteq E(\ell')$, the set enumerated by Worker n is the same as that enumerated by Worker 1. Information is passed from level n down to level 1 in a way that preserves the full history. Suppose Worker n has produced $\sigma \ell^n u$. For $1 \leq k \leq n-1$, Worker k passes this information down, adding to the history its

own most recent ℓ^k, which satisfies $\ell^{k+1} \leq_k \ell^k$. Eventually, Worker 1, having determined ℓ^1, will see the top level sequence $\sigma\ell^n u$ together with the history

$$\ell^n \leq_{n-1} \ell^{n-1} \leq_{n-2} \cdots \leq_2 \ell^2 \leq_1 \ell^1.$$

Worker 1 must find ℓ such that $\sigma\ell^n u\ell$ is in P, and for $1 \leq k < n$, $\ell^k \leq_{k-1} \ell$. Ash's conditions guarantee that such an ℓ exists. Worker 1 can find one since P and the back-and-forth relations are all c.e.

3.2.2 What is new in our proof?

We indicate what is the same and what is different in our proof. For simplicity, we treat the oracle of Worker 1 as computable, although it is actually $\Delta^0_n(T)$. Our objects satisfy the hypotheses of Ash's metatheorem, except that the back-and-forth relations are not all c.e. Our instruction function gives a generic path through T, and $\cup_s E(\ell^n_s)$ is the atomic diagram of a structure $\mathcal{B}$ isomorphic to $\mathcal{A}^p$. Our back-and-forth relations are the symmetric ones, but they satisfy Ash's four conditions. If we try to proceed exactly as in the proof from [2], then as above, we would have $\sigma\ell^n u$, produced by Worker n, and ℓ^k for $1 \leq k < n$, produced by Worker k, such that

$$\ell^n \equiv_{n-1} \ell^{n-1} \equiv_{n-2} \cdots \equiv_2 \ell^2 \equiv_1 \ell^1.$$

Since Ash's four conditions are satisfied, there is some ℓ such that $\sigma\ell^n u\ell \in P$, and $\ell^k \equiv_{k-1} \ell$ for $1 \leq k < n$. The difficulty is that Worker 1 needs to find ℓ, and the relations $\equiv_k$ are not all c.e.

The new idea in our proof is that, instead of just passing down the instruction u, adding to the history, and leaving to Worker 1 the responsibility of determining ℓ, the workers modify the instruction. The relations $\equiv_k$ and $\subseteq$ are defined on all steps—including the modified instructions, which we call *requests*. Worker n transforms the instruction u into a request u^n. Worker $n-1$, having arrived at step ℓ^{n-1} before correctly and stably guessing the steps of Worker n up to $\ell^n u^n$, produces the modified request $u^{n-1} \supseteq u^n$. In general, for $2 \leq k \leq n-1$, Worker k, having arrived at ℓ^k before correctly and stably guessing the sequence of steps of Worker $k+1$ up to u^{k+1}, modifies it to produce $u^k \supseteq u^{k+1}$. For ℓ to fulfill a request u^k means that $\ell \supseteq u^k$. Worker 1, having correctly and stably guessed the steps of Worker 2 up to $\ell^2 u^2$, finds ℓ fulfilling the request u^2, and this also fulfills the requests u^k for $2 < k \leq n$.

In modifying the request u^n, Worker $n-1$ uses just $\equiv_{n-2}$ and $\subseteq$. For $2 \leq k \leq n-2$, Worker k determines u^k using just $\equiv_{k-1}$ and $\equiv_k$. Worker 1 does not issue requests, but uses $\equiv_0$ and $\equiv_1$ in fulfilling the request u^2.[5] This is the essential innovation that allows our construction to work. The symmetric back-and-forth relations were forced on us because of the requests. To indicate why this is, we consider how Worker k determines a request. Say that Worker k has obtained ℓ^k and guessed $\ell^{k+1}u^{k+1}$. We will have $\ell^{k+1} \equiv_k \ell^k$ and $\ell^{k+1} \equiv_k u^{k+1}$. The request u^k is determined in two steps. First, find $\ell^* \supseteq \ell^{k+1}$ such that $\ell^k \equiv_{k-1} \ell^*$. Then find $u^k \supseteq u^{k+1}$ such that $\ell^* \equiv_{k-1} u^k$. We will have $\ell^k \equiv_{k-1} u^k$. If we try to use the asymetric back-and-forth relations, with $\ell^{k+1} \leq_k \ell^k$ and $\ell^{k+1} \leq_k u^{k+1}$, we get stuck trying to find $u^k \supseteq u^{k+1}$ such that $\ell^k \leq_{k-1} u^k$.

[5] In the case $n = 2$, Worker 1 and Worker $n-1$ are the same. In this case, Worker 1 uses $\equiv_0$ and $\subseteq$. Several aspects of the proof below must be modified similarly in the case $n = 2$.

3.2.3 What does each worker do?

What we said above was very general. We must say more precisely what our workers do, given $T \cong T_{\mathcal{A}}$ for $\mathcal{A}$ of ELL-rank at most n, to produce $\mathcal{B} \cong \mathcal{A}$. Worker 1 enumerates the diagram of $\mathcal{B}$. Worker n produces an ELL-n-generic path p through T and an isomorphism F from $\mathcal{B}$ onto $\mathcal{A}^p$. Worker n satisfies the following witnessing requirements.

Requirement $W_{(\nu,\tau)}$: For $\nu \prec \tau$ in T, where ν has variables $\bar{x}$ and τ has variables $\bar{x}, \bar{y}$, if $\nu \prec p$, then ν extends to some $\tau' \prec p$ with variables including $\bar{x}$ and some $\bar{z}$ such that $(\tau, \bar{x}, \bar{y}) \equiv_{n-1} (\tau', \bar{x}, \bar{z})$.

Recall that the universe of $\mathcal{A}^p$ is the set $X = \{x_1, x_2, \dots\}$. The atomic diagram of $\mathcal{A}^p$ is given by the labels on the nodes of the path p. The structure $\mathcal{B}$ is Δ^0_n, while the path p and the isomorphism F are Δ^0_{2n-1} (all uniformly in T). Each worker produces a sequence of *completed steps* of the form $\ell = (\sigma, g)$, where σ is a node of T that might lie on the path p, and g is a finite $1-1$ function that might be a part of F, so that g maps a tuple $\bar{b}$ in the universe of $\mathcal{B}$ to the variables of σ. If σ is at level k of T, then the variables are $x_1, \dots, x_k$. The label on σ assigns an atomic type to $\bar{x}$, and, therefore, to $\bar{b}$. The node σ, when thought of as part of a generic path p, also determines higher-complexity types.

3.2.4 The c.e. set $\mathbb{L}$

Let $\mathbb{L}$ be the set of pairs $\ell = (\sigma, g)$ such that $\sigma \in T$ and g is a finite $1-1$ function mapping some tuple $\bar{b}$ in ω (the universe of $\mathcal{B}$) onto the variables $\bar{x}$ of σ. The elements of $\mathbb{L}$ are the possible completed steps. There is a natural notion of extension on $\mathbb{L}$. For $\ell = (\sigma, g)$ and $\ell' = (\sigma', g')$, we write $\ell \subseteq \ell'$ if $\sigma \preceq \sigma'$ and $g \subseteq g'$. Below, we define back-and-forth relations $\ell \equiv_k \ell'$ on $\mathbb{L}$, with the intended meaning that the $\equiv_k$-class assigned by ℓ to a tuple $\bar{b}$ in the universe of $\mathcal{B}$ matches that assigned by ℓ'. We have already defined relations $(\sigma, \bar{x}) \equiv_k (\tau, \bar{y})$, where $\sigma, \tau \in T$ and $\bar{x}$, $\bar{y}$ are tuples of variables of the same length, with $\bar{x}$ included among the variables of σ and $\bar{y}$ included among the variables of τ. We use these relations to define relations $\equiv_k$ on $\mathbb{L}$.

Definition 3.1. Let $\ell = (\sigma, g)$ and $\ell' = (\sigma', g')$ be elements of $\mathbb{L}$ such that $\mathrm{dom}(g) \subseteq \mathrm{dom}(g')$. Say that $\mathrm{dom}(g) = \bar{b}$, $g(\bar{b}) = \bar{x}$, and $g'(\bar{b}) = \bar{y}$. Then $\ell \equiv_k \ell'$ if $(\sigma, \bar{x}) \equiv_k (\sigma', \bar{y})$.

Remark: Despite the notation, the relation $\equiv_k$ is not symmetric; the right-hand side may involve a function with strictly larger domain.

3.2.5 Issuing and fulfilling requests

Worker 1 produces just the sequence of completed steps. For $2 \leq k \leq n$, Worker k alternates completed steps with "requests," of the form $(\tau, h, \bar{v})$, where $\tau \in T$, h is a finite $1-1$ function taking a tuple $\bar{b}$ from ω to a tuple $\bar{x}$ among the variables of τ, and $\bar{v}$ consists of the remaining variables of τ. In issuing the request, Worker k is asking the worker below for help in finding appropriate pre-images for the variables in $\bar{v}$. Let $\mathbb{U}$ be the set of these possible requests. We broaden our use of $\subseteq$ to include pairs from $\mathbb{U} \times \mathbb{U}$, and our use of the notations

$\subseteq$, $\equiv_k$ to include pairs from $\mathbb{L} \times \mathbb{U}$ and $\mathbb{U} \times \mathbb{L}$. Suppose that $u = (\tau, h, \bar{v})$ and $u' = (\tau', h', \bar{v}')$ are in $\mathbb{U}$, and that $\ell = (\sigma, g) \in \mathbb{L}$.

(1) $u \subseteq u'$ if $\tau \preceq \tau'$ and $h \subseteq h'$.

(2) $\ell \subseteq u$ if $\sigma \preceq \tau$ and $g \subseteq h$, and for $k \leq n-1$, $\ell \equiv_k u$ if $\mathrm{dom}(g) \subseteq \mathrm{dom}(h)$, where g maps $\bar{b}$ to $\bar{x}$, h maps $\bar{b}, \bar{b}'$ to $\bar{y}, \bar{y}'$ such that $(\sigma, \bar{x}) \equiv_k (\tau, \bar{y})$.

(3) $u \subseteq \ell$ if $\tau \preceq \sigma$ and g extends h so as to include $\bar{v}$ in $\mathrm{ran}(g)$. For $k < n$, $u \equiv_k \ell$ if $\mathrm{dom}(h) \subseteq \mathrm{dom}(g)$, where h maps $\bar{b}$ to the tuple $\bar{x}$ consisting of variables of τ not in $\bar{v}$, and g maps $\bar{b}, \bar{b}'$ to $\bar{x}', \bar{v}'$ such that $(\tau, \bar{x}, \bar{v}) \equiv_k (\sigma, \bar{x}', \bar{v}')$.

Definition 3.2. The request $u = (\tau, h, \bar{v}) \in \mathbb{U}$ is "fulfilled" by $\ell = (\sigma, g) \in \mathbb{L}$ if $u \subseteq \ell$, and $\mathrm{ran}(g)$ includes the full tuple of variables of σ.

Note: If $u \subseteq u'$ and ℓ fulfills u', then ℓ also fulfills u.

3.2.6 Back-and-forth relations computed by a given worker

Fact 3.3. Worker k, for $1 \leq k \leq n$, can compute $\equiv_{k-1}$. For $1 \leq k \leq n-2$, Worker k can also compute $\equiv_k$. All of the workers can compute $\subseteq$.

Proof. The relation $\equiv_k$ is Π_{2k}-definable, so it can be decided using oracle Δ^0_{2k+1}. For $1 \leq k \leq n$, Worker k has oracle Δ^0_{n+k-1}. For Worker k to compute $\equiv_{k-1}$ we need $2(k-1)+1 \leq n+k-1$, i.e., $k \leq n$. For Worker k to compute $\equiv_k$ we need $2k+1 \leq n+k-1$, i.e., $k \leq n-2$. □

Worker	Oracle	Computes
n	Δ^0_{2n-1}	$\subseteq$, $\equiv_{n-1}$, ...
$n-1$	Δ^0_{2n-2}	$\subseteq$, $\equiv_{n-2}$, ...
$n-2$	Δ^0_{2n-3}	$\subseteq$, $\equiv_{n-2}$, $\equiv_{n-3}$,...
$\vdots$	$\vdots$	$\vdots$
k	Δ^0_{n+k-1}	$\subseteq$, $\equiv_k$, $\equiv_{k-1}$, ...
$\vdots$	$\vdots$	$\vdots$
2	Δ^0_{n+1}	$\subseteq$,$\equiv_2$, $\equiv_1$, $\equiv_0$
1	Δ^0_n	$\subseteq$,$\equiv_1$, $\equiv_0$

The above table assumes that n is large enough to make the first two rows sensible.

3.2.7 Steps of each worker

The setup of the worker argument is much like that in the proof in [2] of Ash's metatheorem. For $2 \leq k \leq n$, Worker k produces an alternating sequence $\ell^k_0 u^k_1 \ell^k_1 u^k_2 \ldots$, where $\ell^k_m = (\sigma^k_m, g^k_m)$ is a completed step and u^k_{m+1} is the next request. The first completed step is the same for all workers—$\ell^k_0 = (\lambda, \emptyset)$, where λ is the root of T and $\emptyset$ is the empty function. Worker 1 produces just a sequence of completed steps $\ell^1_0 \ell^1_1 \ell^1_2 \ldots$, where $\ell^1_m = (\sigma^1_m, g^1_m)$. It is Worker 1 who builds the structure $\mathcal{B}$. Suppressing the bottom-level oracle, we say that $\mathcal{B}$ is computable—it is actually $\Delta^0_n(T)$. A completed step $\ell^1_s = (\sigma^1_s, g^1_s)$ determines

a map g_s^1 sending a tuple $\bar{b} \in \mathcal{B}$ to the variables of σ_s^1, and σ_s^1 determines the atomic type of these. Worker 1 enumerates the atomic diagram of $\mathcal{B}$. To maintain consistency, Worker 1 must preserve $\equiv_0$ between successive completed steps.

For each k, Worker $k+1$ obtains its completed steps by accepting certain completed steps of Worker k, so the completed steps of Worker $k+1$ form a subsequence of the completed steps of Worker k. The completed steps of Worker n, on top, are the ones of Worker 1 that have been accepted all the way up. Worker n controls the construction of an ELL-n-generic path p through T and an isomorphism F from $\mathcal{B}$ onto $\mathcal{A}^p$. Worker n must preserve the extension relation between successive completed steps. Since Worker n can compute $\equiv_{n-1}$, it can meet the genericity requirements.

Having a completed step $\ell_s^n = (\sigma_s^n, g_s^n)$, Worker n determines an extension σ of σ_s^n satisfying the next "genericity" requirement. Since the structure $\mathcal{B}$ must be Δ_n^0 relative to T, Worker n needs help from the workers below. Worker n makes a request of Worker $n-1$, who will make a request of Worker $n-2$, and so on. Eventually, Worker 1 fulfills the request from Worker 2, in a completed step. This completed step will then be accepted by Worker 2. The same completed step fulfills the request from Worker 3, who accepts it, and by Worker 4, who also accepts it, and so on up until it is accepted by Worker n.

Worker k cannot compute the sequence produced by Worker $k+1$. However, Worker k can approximate this sequence, such that for each finite initial segment, the approximations are eventually correct. Worker k will give some requests to Worker $k-1$ that are based on incorrect guesses, but for each request u of Worker $k+1$, Worker k eventually has a stable correct guess at the sequence of Worker $k+1$ that ends in u. Then Worker k produces a request $u' \supseteq u$. Moreover, the steps of Worker k based on incorrect guesses never go off the rails. While guessing, possibly incorrectly, a longer sequence of Worker $k+1$, Worker k maintains consistency with the initial segments that are correct.

3.2.8 What is preserved, and what is a "reasonable" guess?

Worker n preserves $\subseteq$; we require $\ell_i^n \subseteq u_{i+1}^n \subseteq \ell_{i+1}^n$. For Worker k, where $2 \leq k \leq n-1$, we require $\ell_m^k \equiv_{k-1} u_{m+1}^k \subseteq \ell_{m+1}^k$. For Worker 1, we require $\ell_m^1 \equiv_0 \ell_{m+1}^1$. We will have an additional condition as well: If $\ell_i^n = (\sigma, g)$ and $u_{i+1}^n = (\tau, h, \bar{v})$, then $\mathrm{dom}(g) = \mathrm{dom}(h)$. For $2 \leq k \leq n-1$, Worker k, once it has ℓ_m^k, determines u_{m+1}^k based on a guess at the sequence of steps of Worker $k+1$ up to the first unfulfilled request. Worker 1, once it has ℓ_m^1, determines ℓ_{m+1}^1 based on a guess at the sequence of steps of Worker 2 up to the first unfulfilled request. To avoid fatal mistakes, we require that the guesses be "reasonable."

Suppose Worker k has produced a sequence ending in ℓ_m^k and is guessing at the sequence produced by Worker $k+1$ up to the first unfulfilled request. The guess will have the form $\ell_0^{k+1} u_1^{k+1} \ell_1^{k+1} \ldots \ell_p^{k+1} u_{p+1}^{k+1}$, where $\ell_i^{k+1} = \ell_{m_i}^k$ for some $0 = m_0 < m_1 < \ldots < m_p \leq m$. This indicates that Worker k believes it has fulfilled request u_i^{k+1} in $\ell_{m_i}^k$, and, as a result, $\ell_{m_i}^k$ has been accepted as ℓ_i^{k+1}. For the guess to be "reasonable," we require more.

Definition 3.4.

(1) For $k = n-1$ being *reasonable* means that $\ell_i^n \subseteq \ell_j^{n-1}$ for $m_i \leq j \leq m$. In addition, $\ell_p^n \subseteq u_{p+1}^n$, and if $\ell_i^n = (\sigma, g)$ and $u_{i+1}^n = (\tau, h, \bar{v})$, then

$\mathrm{dom}(g) = \mathrm{dom}(h)$.

(2) For $1 \leq k \leq n-2$ being *reasonable* means that $\ell_i^{k+1} \equiv_k \ell_j^k$ for $m_i \leq j \leq m$. In addition, $\ell_p^{k+1} \equiv_k u_{p+1}^{k+1}$, and if $\ell_i^{k+1} = (\sigma, g)$ and $u_{i+1}^{k+1} = (\tau, h, \bar{v})$, then $\mathrm{dom}(g) = \mathrm{dom}(h)$.

3.2.9 More on requests

We must say exactly how Worker k defines u_{m+1}^k, for $2 \leq k \leq n$.

(1) Given $\ell_m^n = (\sigma, g)$, Worker n locates the first witnessing requirement $W_{(\nu,\tau)}$ that needs attention at σ. Thus $\nu \preceq \sigma$, and, writing $\bar{x}$ for the variables of ν and $\bar{x}, \bar{y}$ for those of τ, there is no tuple $\bar{z}$ among the variables of σ such that $(\tau, \bar{x}, \bar{y}) \equiv_{n-1} (\sigma, \bar{x}, \bar{z})$. Worker n finds an extension $\sigma' \succeq \sigma$, with variables including some $\bar{z}$, such that $(\tau, \bar{x}, \bar{y}) \equiv_{n-1} (\sigma', \bar{x}, \bar{z})$. Let $\bar{v}$ consist of the variables of σ' that are not in the range of g. Then $u_{m+1}^n = (\sigma', g, \bar{v})$.

(2) Given $\ell_m^{n-1} = (\sigma, g)$, Worker $n-1$ guesses the sequence of steps of Worker n up to the first unfulfilled request, checking that the guess is reasonable. Say that this guess ends in $\ell_p^n u_{p+1}^n$, where $\ell_p^n = (\sigma', g')$ and $u_{p+1}^n = (\tau, h, \bar{v})$. By reasonableness, we have $\ell_p^n \subseteq \ell_m^{n-1}$ and $\ell_p^n \subseteq u_{p+1}^n$, and $\mathrm{dom}(g') = \mathrm{dom}(h)$. Say that g' maps $\bar{b}$ to $\bar{x}$; then h also maps $\bar{b}$ to $\bar{x}$. Say that g maps $\bar{b}, \bar{b}'$ to $\bar{x}, \bar{y}$. Choose an extension τ' of τ and a function $h' \supseteq h$ with $\mathrm{dom}(h') = \mathrm{dom}(g)$ such that h' maps $\bar{b}, \bar{b}'$ to $\bar{x}, \bar{z}$ and $(\sigma, \bar{x}, \bar{y}) \equiv_{n-2} (\tau', \bar{x}, \bar{z})$. Let $\bar{v}'$ consist of the variables of τ' that are not in $\mathrm{ran}(h')$. Then the desired request is $u_{m+1}^{n-1} = (\tau', h', \bar{v}')$.

(Why do such τ' and h' exist? Recall that, relative to σ, $\bar{x}$ and $\bar{y}$ represent some elements of $\mathcal{A}$. Since $\sigma' \preceq \sigma$ and $\sigma' \preceq \tau$, relative to σ and τ the tuple $\bar{x}$ represents the same elements. There is some extension of τ in which we find elements automorphic to those represented by $\bar{y}$ over the elements represented by $\bar{x}$. The node τ' is long enough to find such elements. Worker $n-1$ is not able to check automorphism equivalence, but it can check $\equiv_{n-2}$.)

(3) Given $\ell_m^k = (\sigma, g)$, where $2 \leq k \leq n-2$, Worker k guesses the sequence of steps of Worker $k+1$ up to the first unfulfilled request, checking that the guess is reasonable. Say that this guess ends in $\ell_p^{k+1} u_{p+1}^{k+1}$, where $\ell_p^{k+1} = (\sigma', g')$ and $u_{p+1}^{k+1} = (\tau, h, \bar{v})$. By reasonableness, we have $\ell_p^{k+1} \equiv_k \ell_m^k$ and $\ell_p^{k+1} \equiv_k u_{p+1}^{k+1}$, and $\mathrm{dom}(g') = \mathrm{dom}(h)$. Say that g' maps $\bar{b}$ to $\bar{x}$, h maps $\bar{b}$ to $\bar{y}$, and g maps $\bar{b}, \bar{b}'$ to $\bar{z}, \bar{z}'$. Choose an extension τ' of τ and a function $h' \supseteq h$ with $\mathrm{dom}(h') = \mathrm{dom}(g)$ such that h' maps $\bar{b}, \bar{b}'$ to $\bar{y}, \bar{y}'$ and $(\sigma, \bar{z}, \bar{z}') \equiv_{k-1} (\tau', \bar{y}, \bar{y}')$. Let $\bar{v}'$ consist of the variables of τ' that are not in $\mathrm{ran}(h')$. Then the desired request is $u_{m+1}^k = (\tau', h', \bar{v}')$.

(Why do such τ' and h' exist? By reasonableness, $(\sigma', \bar{x}) \equiv_k (\sigma, \bar{z})$ and $(\sigma', \bar{x}) \equiv_k (\tau, \bar{y})$, so $(\tau, \bar{y}) \equiv_k (\sigma, \bar{z})$. One step of the back-and-forth relation gives an extension τ' of τ and a tuple $\bar{y}'$ such that $(\sigma, \bar{z}, \bar{z}') \equiv_{k-1} (\tau', \bar{y}, \bar{y}')$. Worker k can check $\equiv_k$ and $\equiv_{k-1}$.)

(4) Given $\ell_m^1 = (\sigma, g)$, Worker 1 guesses the sequence of steps of Worker 2 up to the first unfulfilled request, checking that the guess is reasonable. Say

that this guess ends in $\ell_p^2 u_{p+1}^2$, where $\ell_p^2 = (\sigma', g')$ and $u_{p+1}^2 = (\tau, h, \bar{v})$. By reasonableness, we have $\ell_p^2 \equiv_1 \ell_m^1$ and $\ell_p^2 \equiv_1 u_{p+1}^2$, and $\mathrm{dom}(g') = \mathrm{dom}(h)$. Say that g' maps $\bar{b}$ to $\bar{x}$, h maps $\bar{b}$ to $\bar{y}$, and g maps $\bar{b}, \bar{b}'$ to $\bar{z}, \bar{z}'$. Choose an extension τ' of τ and a function $h' \supseteq h$ with $\mathrm{dom}(h') = \mathrm{dom}(g)$ such that h' maps $\bar{b}, \bar{b}'$ to $\bar{y}, \bar{y}'$ and $(\sigma, \bar{z}, \bar{z}') \equiv_0 (\tau', \bar{y}, \bar{y}')$. Extend τ' to σ^* and h' to a function g'', if necessary, so that $\mathrm{ran}(g'')$ includes all of the variables of σ^* and $\mathrm{dom}(g'')$ includes the first $m+1$ elements of ω. Then $\ell_{m+1}^1 = (\sigma^*, g'')$ fulfills u_{p+1}^2. (Worker 1 does not issue a request.)

(Why do such τ' and h' exist? By reasonableness, $(\sigma', \bar{x}) \equiv_1 (\sigma, \bar{z})$ and $(\sigma', \bar{x}) \equiv_1 (\tau, \bar{y})$, so $(\tau, \bar{y}) \equiv_1 (\sigma, \bar{z})$. One step of the back-and-forth relation gives an extension τ' of τ and a tuple $\bar{y}'$ such that $(\sigma, \bar{z}, \bar{z}') \equiv_0 (\tau', \bar{y}, \bar{y}')$. Worker 1 can check $\equiv_1$ and $\equiv_0$.)

After issuing a request u_m^k, Worker k must wait until Worker $k-1$ has fulfilled it. Worker $k-1$ will produce a request, based on a correct stable guess at what Worker k has done, and then wait for Worker $k-2$ to fulfill it. Worker $k-2$ will eventually guess what Worker $k-1$ has done and produce a request. The requests form a chain. Eventually, Worker 1 has a stable correct guess at what Worker 2 has done, and fulfills Worker 2's request. This also fulfills the requests of the workers above, so it is accepted as the next completed step for all.

3.2.10 Accepting completed steps

Suppose that Worker k has so far produced completed steps $\ell_0^k, \ldots, \ell_m^k$, and that Worker $k+1$ has accepted the completed steps $\ell_0^{k+1}, \ldots, \ell_p^{k+1}$. Thus, for some $0 = m_0 < \cdots < m_p < m$, we have $\ell_i^{k+1} = \ell_{m_i}^k$ for $0 \leq i \leq p$. Suppose that the current request of Worker $k+1$ is u_{p+1}^{k+1}. We describe when Worker $k+1$ accepts the as-yet-unaccepted completed step ℓ_m^k.

For $k \geq 2$, Worker $k+1$ waits until it sees that, in issuing u_m^k, Worker k used a correct and stable approximation to the oracle of Worker $k+1$; that is, a true stage in the sense of Lachlan. In particular, Worker k must have correctly guessed the sequence $\ell_0^{k+1} u_1^{k+1} \ell_1^{k+1} \cdots \ell_p^{k+1} u_{p+1}^{k+1}$. The request produced from this guess satisfies $u_m^k \supseteq u_{p+1}^{k+1}$, and Worker k accepts ℓ_m^k only when it fulfills u_m^k. Hence $\ell_m^k \supseteq u_m^k \supseteq u_{p+1}^{k+1}$, so ℓ_m^k fulfills the outstanding request of Worker $k+1$. At this point Worker $k+1$ accepts it, setting $\ell_{p+1}^{k+1} = \ell_m^k$. When $k = 1$, the middle request is omitted: after a correct and settled approximation to the oracle of Worker 2, Worker 1 constructs its completed step so as to fulfill u_{p+1}^2 directly.

Whenever Worker $k+1$ accepts a completed step of Worker k, Worker k had a correct and settled approximation to the oracle of Worker $k+1$. Thus, from that point on, Worker k's guess at Worker $k+1$'s completed steps will always begin with

$$\ell_0^{k+1} u_1^{k+1} \ell_1^{k+1} \cdots u_{p+1}^{k+1} \ell_{p+1}^{k+1}.$$

In particular, if $k \leq n-2$, then $\ell_{p+1}^{k+1} \equiv_k \ell_j^k$ for all $j \geq m$; if $k = n-1$, then $\ell_{p+1}^n \subseteq \ell_j^{n-1}$ for all $j \geq m$.

3.2.11 Filling the gaps and completing the proof

We have described the construction somewhat informally, leaving a few gaps. We now fill those gaps and show that the construction succeeds. We must show that for $1 \leq k \leq n-1$, once Worker k has determined ℓ_m^k, there is always a reasonable guess—namely, the truth—on which Worker k can base its next step. We must also show that whenever Worker k acts on a reasonable guess, even if it was wrong, no harm is done.

Lemma 3.5. *Let $1 \leq k \leq n-1$, and suppose that Worker k has determined its sequence of completed steps*

$$\ell_0^k, \ell_1^k, \ldots, \ell_m^k$$

and the true current state of Worker $k+1$ is

$$\ell_0^{k+1} u_1^{k+1} \ell_1^{k+1} \cdots \ell_p^{k+1} u_{p+1}^{k+1},$$

where $\ell_i^{k+1} = \ell_{m_i}^k$ for $0 = m_0 < \cdots < m_p \leq m$.

(1) The true current state of Worker $k+1$, through its first unfulfilled request, is a reasonable guess on which Worker k can base its next action.

(2) Suppose that Worker k acts on a reasonable guess.

(a) If $k = 1$, then the resulting completed step ℓ_{m+1}^1 satisfies

$$\ell_i^2 \equiv_1 \ell_{m+1}^1 \quad \text{for every } i \leq p.$$

(b) If $2 \leq k \leq n-2$, then the resulting request u_{m+1}^k satisfies

$$\ell_i^{k+1} \equiv_k u_{m+1}^k \quad \text{for every } i \leq p$$

and thus the resulting completed step ℓ_{m+1}^k satisfies

$$\ell_i^{k+1} \equiv_k \ell_{m+1}^k \quad \text{for every } i \leq p.$$

(c) If $k = n-1$, then the resulting request u_{m+1}^{n-1} satisfies

$$\ell_i^n \subseteq u_{m+1}^{n-1} \quad \text{for every } i \leq p$$

and thus the resulting completed step ℓ_{m+1}^k satisfies

$$\ell_i^n \subseteq \ell_{m+1}^{n-1} \quad \text{for every } i \leq p.$$

Proof. The proof is by simultaneous induction on the steps of the construction. At the start of the construction, all of these conditions hold. Now suppose that we are at some later point in the construction.

For (1), when $k \leq n-2$, the preservation condition for Worker $k+1$ gives $\ell_p^{k+1} \equiv_k u_{p+1}^{k+1}$, while (2) gives, for each $i \leq p$ and each j with $m_i \leq j \leq m$, $\ell_i^{k+1} \equiv_k \ell_j^k$. The functions in ℓ_i^{k+1} and u_{i+1}^{k+1} have the same domain for every $i \leq p$ because of the way Worker $k+1$ issues its requests. These are exactly the conditions which say that the current true state is reasonable. If $k = n-1$ the argument is the same except that we use $\subseteq$ instead of $\equiv_k$.

For (2), suppose that Worker k acts on a reasonable guess. When Worker $k+1$ accepted the completed true stages through ℓ_p^{k+1}, it was because Worker k used correct and settled approximations to the oracle of Worker $k+1$. Thus every later approximation by Worker k extends the settled true prefix

$$\ell_0^{k+1} u_1^{k+1} \cdots \ell_p^{k+1}.$$

In particular, any later reasonable guess has the form

$$\ell_0^{k+1} \cdots \ell_p^{k+1} \hat{u}_{p+1}^{k+1} \hat{\ell}_{p+1}^{k+1} \cdots \hat{\ell}_{p+q}^{k+1} \hat{u}_{p+q+1}^{k+1}$$

for some $q \geq 0$. By reasonableness, for every $i \leq p$,

$$\ell_i^{k+1} \equiv_k \hat{\ell}_{p+q}^{k+1} \equiv_k \hat{u}_{p+q+1}^{k+1}$$

when $k \leq n-2$, and

$$\ell_i^n \subseteq \hat{\ell}_{p+q}^{k+1} \subseteq \hat{u}_{p+q+1}^n$$

when $k = n-1$. We check the three cases.

(a) If $k = 1$, Worker 1 constructs ℓ_{m+1}^1 so as to fulfill $\hat{u}_{p+q+1}^2$. For every $i \leq p$, $\ell_i^2 \equiv_1 \hat{u}_{p+q+1}^2 \equiv_1 \ell_{m+1}^1$.

(b) If $2 \leq k \leq n-2$, Worker k constructs $u_{m+1}^k \supseteq \hat{u}_{p+q+1}^{k+1}$. Then $\ell_i^{k+1} \equiv_k u_{m+1}^k$ for every $i \leq p$. When u_{m+1}^k is fulfilled by $\ell_{m+1}^k \supseteq u_{m+1}^k$ we have

$$\ell_i^{k+1} \equiv_k \ell_{m+1}^k \quad \text{for every } i \leq p.$$

(c) Finally, if $k = n-1$, Worker $n-1$ constructs $u_{m+1}^{n-1} \supseteq \hat{u}_{p+q+1}^n$. By transitivity,

$$\ell_i^n \subseteq u_{m+1}^{n-1} \quad \text{for every } i \leq p.$$

When this request is fulfilled by $\ell_{m+1}^{n-1} \supseteq u_{m+1}^{n-1}$, transitivity again gives

$$\ell_i^n \subseteq \ell_{m+1}^{n-1} \quad \text{for every } i \leq p.$$

This gives (2). □

We can now argue that the construction works. We first verify that requests are passed down. We claim that, for $2 \leq k \leq n$, whenever ℓ_m^k is defined, u_{m+1}^k is defined. We argue by downward induction on k. For $k = n$, Worker n always constructs a request. Suppose that $2 \leq k < n$ and that the claim holds for Worker $k+1$. If ℓ_m^k is defined and ℓ_p^{k+1} is the true current completed step of Worker $k+1$, then u_{p+1}^{k+1} is defined by the induction hypothesis. By Lemma 3.5(1), the true current state of Worker $k+1$ through this request is reasonable. Hence Worker k finds a reasonable guess and defines u_{m+1}^k.

We next argue that whenever Worker $k+1$ makes a request u_{p+1}^{k+1}, Worker k eventually fulfills it and Worker $k+1$ accepts the resulting completed step. We prove this by induction on k. For the base case, fix a request u_{p+1}^2 of Worker 2. At some point, Worker 1 has a correct and settled guess at the oracle of Worker 2 through u_{p+1}^2. This guess is reasonable by Lemma 3.5(1), so Worker 1 constructs a completed step that fulfills u_{p+1}^2. Because the guess was correct and settled, Worker 2 accepts this completed step.

Now suppose that $2 \leq k \leq n-1$, and inductively that every request made by Worker k is eventually fulfilled by Worker $k-1$ and accepted by Worker k. In particular, whenever Worker k issues a request, it eventually receives and accepts a completed step and can continue to act. Fix a request u_{p+1}^{k+1} of Worker $k+1$. At some point, Worker k has a correct and stable approximation of the state of Worker $k+1$. By Lemma 3.5(1), this is a reasonable guess, and so Worker k produces a request $u_{m+1}^{k} \supseteq u_{p+1}^{k+1}$. By the induction hypothesis, Worker $k-1$ eventually fulfills u_{m+1}^{k}. The resulting completed step of Worker k therefore also fulfills u_{p+1}^{k+1}. Since u_{m+1}^{k} was issued from a correct and settled approximation to Worker $k+1$, Worker $k+1$ accepts that completed step.

Thus the workers always make requests (other than Worker 1) and eventually accept completed steps fulfilling those requests. In particular ℓ_m^k is defined for all $1 \leq k \leq n$ and all m. The sequence $\ell_m^1 = (\sigma_m^1, g_m^1)$ is defined for all m, preserving $\equiv_0$, $\mathrm{dom}(g_m^1)$ is a tuple $\bar{b}$ that includes the first m elements of ω, and the label on σ_m^1 provides the atomic type of $\bar{b}$. Thus, Worker 1 produces a well-defined structure $\mathcal{B}$ with domain equal to ω. We have also seen that $\ell_m^n = (\sigma_m^n, g_m^n)$ is defined for all m, preserving $\subseteq$, and that for some $0 = s_0 < s_1 < \ldots$, $\ell_m^n = \ell_{s_m}^1$. Thus, Worker n produces a path p through T and a function F that is an isomorphism from $\mathcal{B}$ onto $\mathcal{A}^p$. Worker n ensures that this path is ELL-n-generic.

We have described a uniform effective procedure which, for all $\mathcal{A}$ of ELL-rank at most n and $T \cong T_{\mathcal{A}}$, produces a structure $\mathcal{B}$, computable in $\Delta_n^0(T)$, such that $\mathcal{B} \cong \mathcal{A}^p$ for some ELL-n-generic path p through T. □

3.3 The case α a limit ordinal

In this subsection, we prove Theorem 1.1 in Case 3, where α is a limit ordinal.

Case 3 (α a limit ordinal)**.** *For each computable limit ordinal α, there is an effective Δ_α^0 operator that, for all $\mathcal{A}$ of* ELL*-rank at most α, takes copies of $T_{\mathcal{A}}$ to copies of $\mathcal{A}$.*

Proof. For $\mathcal{A}$ of ELL-rank at most α and $T \cong T_{\mathcal{A}}$, we can apply a uniform Δ_α^0 procedure to find an ELL-α-generic path p through T (see Remark 2.17). From p, we pass effectively to $\mathcal{A}^p$. If $T \cong T_{\mathcal{A}}$, then this $\mathcal{A}^p$ is isomorphic to $\mathcal{A}$. □

3.4 $\alpha = \lambda + n$ for λ a limit ordinal and $n \geq 1$

In this subsection, we complete the proof of Theorem 1.1, with Case 4, where $\alpha = \lambda + n$ for λ a limit ordinal and n finite, with $n \geq 1$.

Case 4. *If $\alpha = \lambda + n$, where λ is a computable limit ordinal and $n \geq 1$ is finite, then there is an effective Δ_α^0 operator that, for $\mathcal{A}$ of* ELL*-rank at most α, takes each $T \cong T_{\mathcal{A}}$ to a copy of $\mathcal{A}$.*

Sketch. We imitate the proof of Case 1 if $n = 1$, and of Case 2 if $n \geq 2$. The imitation of Case 1 is straightforward. For Case 2, we use n workers. For $1 \leq k \leq n$, Worker k has oracle $\Delta_k^0(\Delta_{\lambda+n}^0(T)) = \Delta_{\lambda+n+k-1}^0(T)$. This ensures that the analog of Fact 3.3 holds. Worker n produces an ELL-α-generic path p and an isomorphism F from $\mathcal{B}$ onto $\mathcal{A}^p$. The important back-and-forth relations are between pairs $(\sigma, \bar{x})$, where $\sigma \in T$ and $\bar{x}$ is among the variables of σ. The

completed steps of Worker k have form $\ell_s^k = (\sigma_s^k, g_s^k)$, where g_s^k maps a tuple from $\mathcal{B}$ to the variables $\bar{x}_s^k$ of σ_s^k. Worker k preserves $\equiv_{\lambda+k-1}$; that is, for $\bar{x}$ included among the variables of σ_s^k, $(\sigma_s^k, \bar{x}) \equiv_{\lambda+k-1} (\sigma_{s+1}^k, \bar{x})$. Worker 1 builds $\mathcal{B}$. Worker 1 makes sure that the first s elements of $\mathcal{B}$ are included in $\mathrm{dom}(g_s^1)$. To determine the atomic type of the tuple consisting of the first s elements of $\mathcal{B}$, Worker 1 simply looks at the label on σ_s^1. □

4 Hardness

By relativizing the main result of the previous section, we obtain that for each X-computable ordinal $\alpha \geq 1$, there is an X-effective Δ_α^0 operator that for all $\mathcal{A}$ of Scott rank at most α, takes copies of $T_\mathcal{A}$ to copies of $\mathcal{A}$. We believe that this is optimal; that is, for an ordinal $\alpha \geq 2$, and $1 \leq \beta < \alpha$, there is no (boldface) $\mathbf{\Delta}_\beta^0$ operator that, for all $\mathcal{A}$ of Scott rank at most α, takes copies of $T_\mathcal{A}$ to copies of $\mathcal{A}$. The only method we know for showing non-existence of the $\mathbf{\Delta}_\beta^0$ operator is, fixing a parameter X making the operator X-effective Δ_β^0, to give a structure $\mathcal{A}$ of Scott rank at most α such that $T_\mathcal{A}$ has an X-computable copy while $\mathcal{A}$ has no copy Δ_β^0 relative to X. We can carry this out for finite levels.

Theorem 4.1. *For each $X \subseteq \mathbb{N}$ and $n \in \mathbb{N}$, there is a structure $\mathcal{A}$ of Scott rank $n+2$ such that $T_\mathcal{A}$ has an X-computable copy but $\mathcal{A}$ has no $X^{(n)}$-computable copy. Thus, there is no (boldface) $\mathbf{\Delta}_{n+1}^0$ operator taking copies of $T_\mathcal{A}$ to copies of $\mathcal{A}$ working uniformly on all structures $\mathcal{A}$ of Scott rank at most $n+2$.*

Working relative to X everywhere, we can suppress X. The proof is essentially just putting together two ingredients. Harrison-Trainor and Montalbán [13] gave an example of a structure $\mathcal{B}$ such that $T_\mathcal{B}$ has a computable copy, but $\mathcal{B}$ has no computable copy. On inspection, this structure has Scott rank 2. The construction can be relativized to give a structure $\mathcal{B}$ such that $T_\mathcal{B}$ has a $0^{(2n)}$-computable copy but $\mathcal{B}$ has no $0^{(2n)}$-computable copy. For simplicity we may assume that $\mathcal{B}$ is a graph.

The other ingredient is the finite unfriendly jump inversions introduced by Harrison-Trainor in [11]. From the structure $\mathcal{B}$, we can construct the nth unfriendly jump inversion $\mathcal{B}^{(-n)}$ of $\mathcal{B}$. We will take $\mathcal{A} = \mathcal{B}^{(-n)}$. We will need the formal definition only to prove one fact below.

Definition 4.2 (Harrison-Trainor)**.** Let $S \subseteq \mathbb{N}$ be a complete Σ_{2n+1}^0 set. We fix computable structures $\mathcal{M} \ncong \mathcal{N}$ and a uniformly computable sequence of computable structures $\mathcal{C}_i$ such that

(1) For each i,
$$\mathcal{C}_i \cong \begin{cases} \mathcal{M} & \text{if } i \in S \\ \mathcal{N} & \text{otherwise} \end{cases}$$

(2) $\mathcal{N} \not\leq_{n+1} \mathcal{M}$.

(3) $\mathcal{M}$ and $\mathcal{N}$ have Π_{n+2} Scott sentences.

Such structures exist, as shown in [11]. Let $\mathcal{G}$ be a graph.[6] Define $\mathcal{G}^{(-n)}$ to be the structure consisting of the vertices of $\mathcal{G}$, and associated to each unordered pair

[6]We could, with simple modifications, deal with a structure in any countable relational language.

$\{u, v\}$ of distinct vertices, an ordered pair of structures isomorphic to $(\mathcal{M}, \mathcal{N})$ if there is an edge $u - v$, and isomorphic to $(\mathcal{N}, \mathcal{M})$ if there is no edge $u - v$. More formally, let $\mathcal{L}$ be the language of $\mathcal{M}$ and $\mathcal{N}$. We work in the language $\mathcal{L}^* = \mathcal{L} \cup \{U, E, F\}$ where U is a unary relation and E and F are ternary relations. $\mathcal{G}^{(-n)}$ will be the structure defined as follows. The domain of $\mathcal{G}^{(-n)}$ will consist of the elements of $\mathcal{G}$ together with, for each pair of distinct $u, v \in \mathcal{G}$, disjoint infinite sets $C_{\{u,v\}}$ and $D_{\{u,v\}}$. U will hold only of the elements of $\mathcal{G}$. For distinct u, v, we put $E(u, v, x)$ if and only if $x \in C_{\{u,v\}}$ and $F(u, v, x)$ if and only if $x \in D_{\{u,v\}}$; otherwise, E and F do not hold. The relations of $\mathcal{L}$ will hold only of tuples of elements all in the same set $C_{\{u,v\}}$ or $D_{\{u,v\}}$, and we will have $C_{\{u,v\}} \cong \mathcal{M}$ and $D_{\{u,v\}} \cong \mathcal{N}$ if there is an edge connecting u and v, and $C_{\{u,v\}} \cong \mathcal{N}$ and $D_{\{u,v\}} \cong \mathcal{M}$ if there is no such edge.

We will use the following properties of this jump inversion.

Theorem 4.3 (Harrison-Trainor [11])**.**

(1) Given a $\mathbf{0}^{(2n)}$-computable copy of $\mathcal{B}$, there is a computable copy of $\mathcal{B}^{(-n)}$.

(2) Given a $\mathbf{0}^{(n)}$-computable copy of $\mathcal{B}^{(-n)}$, there is a $\mathbf{0}^{(2n)}$-computable copy of $\mathcal{B}$.

(3) If $\mathcal{B}$ has a Π_ℓ Scott sentence for $\ell \geq 2$, then $\mathcal{B}^{(-n)}$ has a $\Pi_{n+\ell}$ Scott sentence.

In particular, since $\mathcal{B}$ has Scott rank 2; i.e., it has a Π_3 Scott sentence, $\mathcal{A} = \mathcal{B}^{(-n)}$ has a Π_{n+3} Scott sentence and hence has Scott rank at most $n + 2$. If $\mathcal{A} = \mathcal{B}^{(-n)}$ had a $0^{(n)}$-computable copy, then $\mathcal{B}$ would have a $0^{(2n)}$-computable copy; since $\mathcal{B}$ does not have a $0^{(2n)}$-computable copy, $\mathcal{A} = \mathcal{B}^{(-n)}$ does not have a $0^{(n)}$-computable copy.

Finally, we have a $0^{(2n)}$-computable copy of $T_{\mathcal{B}}$ and must argue that there is a computable copy of $T_{\mathcal{A}} = T_{\mathcal{B}^{(-n)}}$. This is essentially the unfriendly analogue of Lemma 6.3 of [13]; intuitively, $T_{\mathcal{B}^{(-n)}}$ and ${T_{\mathcal{B}}}^{(-n)}$ are not too far apart, and (1) of Theorem 4.3 implies that $T_{\mathcal{A}}$ (which is morally ${T_{\mathcal{B}}}^{(-n)}$) has a computable copy. The following lemma makes this formal.

Lemma 4.4. *For any structure $\mathcal{B}$, if $T_{\mathcal{B}}$ has a $0^{(2n)}$-computable copy then $T_{\mathcal{B}^{(-n)}}$ has a computable copy.*

Proof. Assume for simplicity that $\mathcal{B}$ is a graph. Construct computable sequences of structures $\mathcal{C}_{\sigma,\tau}$ and $\mathcal{D}_{\sigma,\tau}$, where $\sigma, \tau \in T_{\mathcal{B}}$ are compatible nodes, such that $(\mathcal{C}_{\sigma,\tau}, \mathcal{D}_{\sigma,\tau}) \cong (\mathcal{M}, \mathcal{N})$ if in $T_{\mathcal{B}}$ there is an edge between (the elements of $\mathcal{B}$ represented by the last entries of) σ and τ, and $(\mathcal{C}_{\sigma,\tau}, \mathcal{D}_{\sigma,\tau}) \cong (\mathcal{N}, \mathcal{M})$ otherwise. In this and what follows, we treat σ, τ as an unordered pair.

The underlying tree of our computable copy of $T_{\mathcal{B}^{(-n)}}$ will be the tree of tuples $\langle b_1, \ldots, b_\ell \rangle$ where:

(1) each b_i is either of the form $\langle \sigma, e \rangle$ where σ is a non-root node in $T_{\mathcal{B}}$ and $e \in \omega$, or is of the form $\langle \sigma, \tau; p, r; e \rangle$ for σ, τ distinct compatible nodes in $T_{\mathcal{B}}$, $p \in \{0, 1\}$, $e \in \omega$, and $r \in \mathcal{C}_{\sigma,\tau}$ if $p = 0$ and $r \in \mathcal{D}_{\sigma,\tau}$ if $p = 1$;

(2) all of the nodes in $T_{\mathcal{B}}$ appearing in $b_1, \ldots, b_\ell$, including the nodes σ, τ when $b_i = \langle \sigma, \tau; p, r; e \rangle$, are from the same path in $T_{\mathcal{B}}$;

(3) among $b_1, \ldots, b_\ell$ there is no repetition of the form $\langle \sigma, e \rangle$ and $\langle \sigma, e' \rangle$ or $\langle \sigma, \tau; p, r; e \rangle$ and $\langle \sigma, \tau; p, r; e' \rangle$.

The b_i of the form $\langle \sigma, e \rangle$ represent elements of $\mathcal{B}$, and nodes $\langle \sigma, \tau; p, r; e \rangle$ represent the rth element of one of the structures $\mathcal{C}_{\sigma,\tau}$ or $\mathcal{D}_{\sigma,\tau}$ attached to (the elements of $\mathcal{B}$ represented by the last entries of) σ and τ. Essentially, we have "mixed in" the elements of the $\mathcal{C}_{\sigma,\tau}$ and $\mathcal{D}_{\sigma,\tau}$. The e are simply to make sure that everything appears infinitely many times as in the definition of the tree of tuples.

Now, given a node $\langle b_1, \ldots, b_\ell \rangle$ of the proposed copy of $T_{\mathcal{B}^{(-n)}}$, we must computably determine the atomic diagram in the language $\mathcal{L}^* = \mathcal{L} \cup \{U, E, F\}$ of the elements of $\mathcal{B}^{(-n)}$ they represent. This is easy to do. Given $b_{i_1}, \ldots, b_{i_m}$ with b_{i_t} of the form $\langle \sigma, \tau; p, r_t; e_t \rangle$ (for fixed σ, τ, p) we put the relations of $\mathcal{L}$ on them according to how they are on $r_1, \ldots, r_m$ in $\mathcal{C}_{\sigma,\tau}$ (if $p = 0$) or $\mathcal{D}_{\sigma,\tau}$ (if $p = 1$). We put U only on those b_i which are of the form $\langle \sigma, e \rangle$, representing an element of $\mathcal{B}$. And we put $E(b_i, b_j, b_k)$ only when $b_i = \langle \sigma, e \rangle$ and $b_j = \langle \tau, f \rangle$ are nodes in $T_{\mathcal{B}}$ and $b_k = \langle \sigma, \tau; 0, r; g \rangle$ and $F(b_i, b_j, b_k)$ only when $b_i = \langle \sigma, e \rangle$ and $b_j = \langle \tau, f \rangle$ are nodes in $T_{\mathcal{B}}$ and $b_k = \langle \sigma, \tau; 1, r; g \rangle$. □

It follows that for all finite $n \geq 1$, there is no $\mathbf{\Delta}^0_n$ operator that, for all $\mathcal{A}$ of Scott rank at most ω, takes copies of $T_{\mathcal{A}}$ to copies of $\mathcal{A}$. Hence, the first case where we do not have a proof of sharpness is $\omega + 1$. There is an effective $\Delta^0_{\omega+1}$ operator that, for all $\mathcal{A}$ of Scott rank at most $\omega + 1$, takes copies of $T_{\mathcal{A}}$ to copies of $\mathcal{A}$, and we do not know whether there is a $\mathbf{\Delta}^0_\omega$ operator.

Question 4.5. Given $\alpha \geq \omega + 1$, does there exist $\beta < \alpha$ and a $\mathbf{\Delta}^0_\beta$ operator that takes copies of $T_{\mathcal{A}}$ to copies of $\mathcal{A}$ for all $\mathcal{A}$ with Scott rank at most α?

Andrews, Gonzalez, and Zhu [1] have extended unfriendly jump inversions through the infinite ordinals. However, this is not sufficient for our purposes. The issue, as discussed in that paper, stems from the fact that for infinite α, $2 \cdot \alpha \neq \alpha \cdot 2$.

References

[1] Uri Andrews, David Gonzalez, and Hongyu Zhu. Infinite belligerent jump inversion and computable scott analysis. Preprint, arXiv:2607.10935.

[2] C.J. Ash and J. Knight. *Computable Structures and the Hyperarithmetical Hierarchy.* Elsevier Science, 2000.

[3] Riccardo Camerlo and Su Gao. The completeness of the isomorphism relation for countable Boolean algebras. *Transactions of the American Mathematical Society*, 353(2):491–518, 2001.

[4] Ruiyan Chen, David Gonzalez, and Matthew Harrison-Trainor. Optimal syntactic definitions of back-and-forth types. Preprint, arXiv:2505.00893.

[5] A. Ehrenfeucht. An application of games to the completeness problem for formalized theories. *Fund. Math.*, 49:129–141, 1960/61.

[6] Harvey Friedman and Lee Stanley. A Borel reducibility theory for classes of countable structures. *J. Symbolic Logic*, 54(3):894–914, 1989.

[7] Su Gao. Some dichotomy theorems for isomorphism relations of countable models. *J. Symbolic Logic*, 66(2):902–922, 2001.

[8] D. Gonzalez, M. Lelyk, D. Rossegger, and P. Szlufik. Classifying the complexities of models of arithmetic. *Submitted for publication. https://arxiv.org/abs/2507.12025.*

[9] David Gonzalez and Julia F. Knight. Computable Scott sentences and the Friedman-Stanley embedding of graphs in labeled trees. Preprint, arXiv: 2605.04404.

[10] David Gonzalez and Dino Rossegger. Scott sentence complexities of linear orderings. *J. Symb. Log.*, 90(4):1563–1592, 2025.

[11] Matthew Harrison-Trainor. Unfriendly jump inversions. Preprint, arXiv: 2505.23613.

[12] Matthew Harrison-Trainor and J. Thomas Kim. Scott spectral gaps for trees are bounded. Preprint available at arxiv.org/pdf/2602.07166.

[13] Matthew Harrison-Trainor and Antonio Montalbán. The tree of tuples of a structure. *J. Symb. Log.*, 87(1):21–46, 2022.

[14] Matthew Harrison-Trainor and Antonio Montalbán. Errata to: The tree of tuples of a structure. `https://homepages.math.uic.edu/~mht/papers/errata-tree-tuples.pdf`, 2026.

[15] Carol R. Karp. Finite-quantifier equivalence. In *Theory of Models (Proc. 1963 Internat. Sympos. Berkeley)*, pages 407–412. North-Holland, Amsterdam, 1965.

[16] H. Läuchli and J. Leonard. On the elementary theory of linear order. *Fund. Math.*, 59:109–116, 1966.

[17] I. A. Lavrov. Effective inseparability of the set of identically true formulae and finitely refutable formulae for certain elementary theories. *Algebra and Logic*, 2:5–18, 1963.

[18] A. Montalbán and D. Rossegger. The structural complexity of models of arithmetic. *The Journal of Symbolic Logic*, First View:1–17, 2023.

[19] Antonio Montalbán. A robuster Scott rank. *Proc. Amer. Math. Soc.*, 143(12):5427–5436, 2015.

[20] Dana Scott. Logic with denumerably long formulas and finite strings of quantifiers. In *Theory of Models (Proc. 1963 Internat. Sympos. Berkeley)*, pages 329–341. North-Holland, Amsterdam, 1965.